AC1 | Cheat Sheet MIPS

Guia de revisão para o teste prático • Baseado nos exercícios treinados e nos erros mais frequentes

Regra principal: lê o C, faz o mapa de registos, identifica os ciclos e só depois escreve o interior. Mantém a ordem do C e comenta a tradução, como pede o teste exemplo.

1. Instruções essenciais

Comando	Significado	Exemplo
li	Coloca um número num registo	li $t0,0
la	Carrega um endereço	la $t0,array
move	Copia entre registos	move $t1,$v0
lw / sw	Lê / escreve inteiro na memória	lw $t1,0($t0)
lb	Lê 1 byte	lb $t1,0($t0)
addiu / addu	Soma constante / 2 registos	addiu $t0,$t0,1
sll	Desloca bits; por 2 = vezes 4	sll $t2,$t1,2
beq / bne	Salta se igual / diferente	beq $t0,$zero,end
bge / ble / blt	Salta >= / <= / <	bge $t0,10,end
j	Salto incondicional	j for

Diferença que não podes trocar: la = endereço; lw = valor da memória; sw = escrever na memória. move só copia registos.

2. Mapa de registos: endereço versus valor

```
la $t0,array      # $t0 = endereço do array
lw $t1,0($t0)     # $t1 = array[0]
sw $t1,4($t0)     # array[1] = $t1
```

O registo dentro de 0($t0) deve conter um endereço. Nunca uses como endereço um registo onde guardaste um valor lido do array. Não alteres o registo do índice com sll $t1,$t1,2; calcula o deslocamento noutro registo.

3. Arrays com índices: i × 4

```
la $t0,array       # início do array
# $t1 = índice i
sll $t2,$t1,2      # i * 4 bytes
addu $t3,$t0,$t2   # &array[i]
lw $t4,0($t3)     # array[i]
lw $t5,4($t3)     # array[i+1]
```

Cada int ocupa 4 bytes. O 2 em sll significa deslocar 2 bits, isto é, multiplicar por 4; não significa avançar duas posições. array[i+4] fica a 16($t3).

4. Trocar elementos com lw + sw

```
# $t6 = &array[i]; $t8 = &array[j]
lw $t9,0($t6)     # valor antigo de array[i]
lw $s0,0($t8)     # valor antigo de array[j]
sw $s0,0($t6)     # array[i] = array[j]
sw $t9,0($t8)     # array[j] = antigo array[i]
```

Não troques com dois move seguidos: perdes o valor original. Para alterar um array na memória, usa sw.

5. Condições if: saltar com a condição contrária

C: executar se...	Para ignorar o if, saltar se...
a > b	ble a,b,endif
a < b	bge a,b,endif
a == b	bne a,b,endif
a != b	beq a,b,endif
a >= b	blt a,b,endif
a <= b	bgt a,b,endif

```
# C: if (a > b) a = b;
ble $t0,$t1,endif
move $t0,$t1
endif:
```

6. Onde colocar os ciclos e incrementos

for: condição no início, incremento antes de voltar. while: testa antes do corpo. do-while: testa no fim, depois de executar o corpo pelo menos uma vez.

```
# for (i=0; i<8; i++)
li $t0,0
for:
    bge $t0,8,endfor
    # corpo
    addiu $t0,$t0,1
    j for
endfor:

# do { i++; } while (i<8);
do:
    addiu $t0,$t0,1
    blt $t0,8,do
```

Dois ciclos: o índice interior incrementa antes de j forin; o exterior incrementa em endforin:. endif: é apenas o fim do if, não o fim de um for.

```
forex:
    bge $t0,3,endforex
    addiu $t1,$t0,1       # j = i+1
forin:
    bge $t1,4,endforin
    # comparação / eventual troca
endif:
    addiu $t1,$t1,1       # j++
    j forin
```

```
endforin:
    addiu $t0,$t0,1       # i++
    j forex
endforex:
```

7. Índices versus ponteiros

Índice i	Ponteiro p (int*)
i++ -> addiu $t1,$t1,1	p++ -> addiu $t3,$t3,4
Endereço: sll e addu	O registo já guarda o endereço
array[i] -> lw 0(endereço)	*p -> lw 0(ponteiro)

```
# C: print_int10(*p++);
lw $a0,0($t3)
li $v0,1
syscall
addiu $t3,$t3,4
```

O incremento de *p++ é do ponteiro, não do valor lido. Se p for char*, costuma avançar 1 byte e usar lb.

8. Impressão de array por ponteiros

```
# $t2 = endereço inicial; SIZE = 8 -> 32 bytes
move $t3,$t2
addiu $t7,$t2,32
while:
    bgeu $t3,$t7,endw
    lw $a0,0($t3)
    li $v0,1
    syscall
    addiu $t3,$t3,4
    li $a0,','
    li $v0,11
    syscall
    j while
endw:
```

Erro recorrente: esquecer addiu ponteiro,ponteiro,4 e j while; nesse caso só imprime o primeiro elemento.

9. Syscalls

$v0	Operação	Como usar
1	Imprimir inteiro	move $a0,$t0; li $v0,1; syscall
4	Imprimir string	la $a0,msg; li $v0,4; syscall
5	Ler inteiro	li $v0,5; syscall; move $t0,$v0
11	Imprimir carácter	li $a0,','; li $v0,11; syscall

10. Diretivas na memória

```
.eqv SIZE,8
.data
array1: .word 8,4,15,2   # 4 inteiros já definidos
array2: .space 32         # espaço para 8 inteiros
msg:    .asciiz "Ola"    # texto terminado em zero
.text
.globl main
```

.word: valores já definidos. .space: reserva bytes para preencher mais tarde. .asciiz: texto; a syscall 4 usa o seu endereço com la.

11. Padrões dos exercícios do teste exemplo

Ex. 1 - Máximo e mínimo: iniciar min = 0x7FFFFFFF e max = 0x80000000; ler com syscall 5; ignorar zero na comparação; dentro do do-while atualizar max/min; incrementar n; continuar enquanto n < 20 e val != 0; imprimir max, ":", min.

Ex. 2 - Troca de metades: com SIZE=8, trocar val[i] e val[i+4]. Se tens endereço de val[i], o outro está em 16(endereço). Guardar os dois valores com lw e trocar com sw. Depois imprimir 8 inteiros com ponteiro, avançando 4 bytes.

Ex. 3 - Ímpares: criar a[35] e b[35] com .space 140 cada. Primeiro for: ler para *p1. Segundo: reiniciar p1=a e p2=b; p1 avança sempre; p2 avança só nos ímpares. Terceiro: imprimir os n_odd elementos de b.

```
# No exercício 3: verificar impar e copiar
lw $t7,0($t2)
andi $t8,$t7,1
beq $t8,$zero,else
sw $t7,0($t3)
addiu $t3,$t3,4      # p2++: só ímpares
addiu $t1,$t1,1      # n_odd++
j endif
else:
addiu $t0,$t0,1      # n_even++
endif:
addiu $t2,$t2,4      # p1++: todos
```

12. Checklist de último minuto

☐ Ler o código C todo e identificar as variáveis.

☐ Criar um mapa de registos consistente.

☐ Separar .data (arrays/strings) de .text (instruções).

☐ Inicializar cada variável no momento correspondente do C.

☐ Condições: saltar com a condição oposta ao if.

☐ for: incremento no fim; do-while: condição no fim.

☐ Índice +1; ponteiro para int +4.

☐ la para endereço; lw para ler; sw para guardar.

☐ Separar endif, endforin e endforex.

☐ Conferir cada etiqueta e testar no MARS.

☐ Respeitar a sequência do C e comentar o Assembly com C.